LECTURE NOTES

Lecture 11: Optimization in Machine Learning

University

Simon Holm
March - 2026

SDU
DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

1 Introduction 3
1.1 Text Classification via Convex Optimization 3
1.2 Formalization 3
1.3 Deep Neural Networks 3
2 Fundamentals 5
2.1 Choice of Prediction Function 5
3 Simplified Notation 5
4 Convex function 6
5 Stochastic vs Batch Optimization Methods 6
5.1 Stochastic Gradient 7
5.2 Beyond SG: Noise Reduction and Second-Order Methods 7
6 Dynamic sample size methods 8
6.1 Guidelines 8
7 Gradient Aggregation 8
7.1 iterated Averaging Methods 9

1 Introduction

Large-scale machine learning represents an important application area of optimization.

Two case studies:

- Logistic regression or support vector machines convex optimization problems
- deep neural networks highly nonlinear and nonconvex problems

1.1 Text Classification via Convex Optimization

Task: determining whether a text document is one that discusses politics.

- Set of examples $\{(x_1, y_1), ..., (x_n, y_n)\}$ where each $i \in \{1, ..., n\}$. x_1 represents the features of a text document (e.g., the words it includes) y_i is a label indicating whether the document belongs $(y_i = 1)$ or not $(y_i = -1)$ to a particular class

Wi minimize the **empirical risk**

$$R_n(h) = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}[h(x_i) \neq y_i], \quad \text{where } \mathbb{I}[A] = \begin{cases} 1 \text{ if } A \text{ is true,} \\ 0 \text{ otherwise} \end{cases}$$

1.2 Formalization

$$h(x; w, \tau) = w^T x + \tau, w \in \mathbb{R}^d \text{ and } \tau \in \mathbb{R}$$

This is the linear discriminator.

performance measure: count how many times $\text{sign}(h(x; w, \tau))$ mispredicts. Discontinuous problem.

alternatively, define a continuous **loss function** ℓ that measures a cost for predicting h when the true label is y; e.g., one may choose a log-loss function of the form $\ell(h, y) = \log(1 + \exp(-hy))$.

We then minimize the empirical risk

$$\min_{(2,\tau) \in \mathbb{R}^d \times \mathbb{R}} \frac{1}{n} \sum_{i=1}^{n} \ell(h(x; w, \tau), y_i) + \frac{\lambda}{2} \|w\|_2^2$$

Then solve for various λ (on the validation set) to the best one.

1.3 Deep Neural Networks

Deep Neural Networks: represent hypotheses as computation graphs with tunable weights and compute the gradient of the loss function with respect to those weights in order to fit the training data.

- Forward accumulation: compute prediction
- Backward accumulation: compute gradient

playground in tensorflow link

2 Fundamentals

One should seek to find h that yields a small expected risk of misclassification over all possible inputs, i.e., an h that minimizes

$$R(h) = P[h(x) \neq y] = \mathbb{E}[\mathbb{I}[h(x) \neq y]]$$

which is **variational** since we are optimizing over a set of functions (the h), and is stochastic since the objective function involves an expectation

2.1 Choice of Prediction Function

$\mathcal{H}$ should contain prediction functions that are able to achieve a low empirical risk over the training set, so as to avoid bias or underfitting the data.

We want to decrease the gap between $\underbrace{R(h)}_{\text{exp risk}} - \underbrace{R_n(h)}_{\text{emp risk}}$

Uniform laws of large numbers and the Hoeffding inequality guarantee that with probability at least

$$1 - \eta$$

$$\sup_{h \in \mathcal{H}} |R(h) - R_n(h)| \leq \mathcal{O}\left(\sqrt{\frac{1}{2n}\log\left(\frac{2}{\eta}\right) + \frac{d_{\mathcal{H}}}{n}\log\left(\frac{n}{d_{\mathcal{H}}}\right)}\right)$$

3 Simplified Notation

Let ξ be a random seed or the realization of a single (or a set of) sample (x, y) For a given (w, ξ) let $f(w; \xi)$ be the composition of the loss function ℓ and the prediction function h Then:

$$\underbrace{R(w) = \mathbb{E}_{\xi}[f(w; \xi)]}_{\text{Expected Risk}}$$

Let $\left\{\xi_{[i]}\right\}_{i=1}^{n}$ be realizations of ξ corresponding to $\{(x_i, y_i)\}_{i=1}^{n}$ and $f_i(w) \overset{\text{def}}{=} f\left(w; \xi_{[i]}\right)$

Then:

$$\underbrace{R_n(w) \overset{\text{def}}{=} \frac{1}{n}\sum_{i=1}^{n} f_i(w)}_{\text{Empirical Risk}}$$

4 Convex function

Proposition

If $[f_j]_{j\in\{1,\dots,m\}}$ are convex and $[]\alpha_j]_{j\in\{1,\dots,m\}}$ are nonnegative, then $\sum_{j=1}^{m}\alpha_j f_j$ and $\max_{j\in\{1,\dots,m\}} f_j$ are convex.

Proposition

If $f:\mathbb{R}^d\to\mathbb{R}$ is convex and $A:\mathbb{R}^d\to\mathbb{R}^d$ is affine then $f\circ A:\mathbb{R}^d\to\mathbb{R}$ is convex.

Proposition

If $f:\mathbb{R}^{d_1+d_2}\to\mathbb{R}$ is convex, so is the function $x_1\mapsto \inf_{x_2\in\mathbb{R}^{d_2}} f(x_1,x_2)$ on $\mathbb{R}^{d_1}$.

Figure 1: The class of convex functions satisfies the following stability properties:

5 Stochastic vs Batch Optimization Methods

We want to minimize R_n with $w_0\in\mathbb{R}^d$. Deterministic problem

Stochastic Approach: Stochastic Gradient

$$w_{k+1}\leftarrow w_k-\alpha_k\nabla f_{i_k}(w_k)$$

i_k is chosen randomly from $\{1,\dots,n\}$, $\alpha_k>0$

- very cheap iteration only on one sample.
- the direction might not always be a descent but if it is a descent direction in **expectation**, then the sequence $\{w_k\}$ can be guided toward a minimizer of R_n.

Batch Approach: batch gradient, steepest descent, full gradient method:

$$w_{k+1}\leftarrow w_k-\alpha_k\nabla R_n(w_k)=w_k-\frac{\alpha_k}{n}\sum_{i=1}^{n}\nabla f_i(w_k)$$

- more expensive
- can use all deterministic gradient-based optimization methods
- the sum structure opens up to parallelization

5.1 Stochastic Gradient

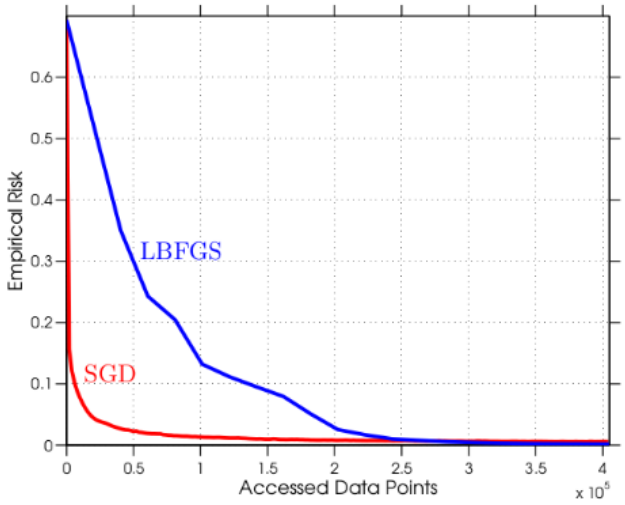

Figure 2: the fast initial improvement achieved by SG, followed by a drastic slowdown after 1 or 2 epochs, is common in practice.

SG more sensitive to α_k and starting point. If more epochs, batch may become better

5.2 Beyond SG: Noise Reduction and Second-Order Methods

$$w_{k+1} \leftarrow w_k - \frac{\alpha_k}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}} \nabla f_{i_k}(w_k)$$

$$F) \begin{cases} R(w) = \mathbb{E}_\xi[f(w;\xi)] & \text{Empirical Risk} \\ R_n(w) = \frac{1}{n}\sum_{i=1}^{n} f_i(w) & \text{Expected Risk} \end{cases}$$

$$g(w_k, \xi_k) = \begin{cases} \nabla f((w_k;\xi_k)) \\ \frac{1}{n_k}\sum_{i=1}^{n_k} \nabla f((w_k;\xi_k)) \\ H_k \cdot \frac{1}{n_k}\sum_{i=1}^{n_k} \nabla f((w_k;\xi_k)) \end{cases}$$

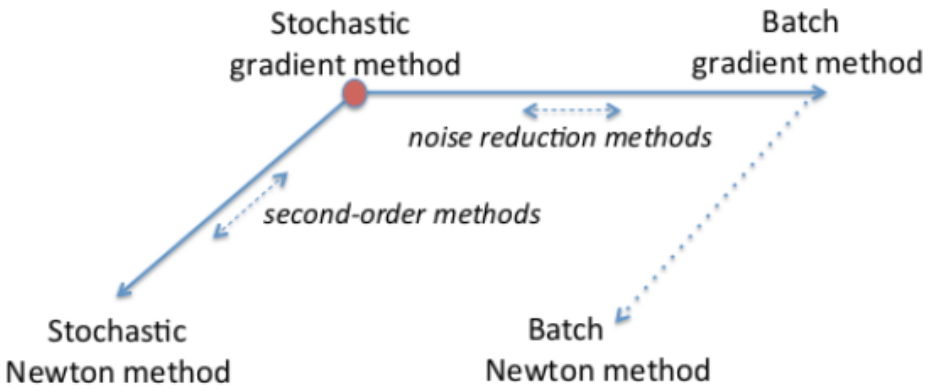

Figure 3: on horizontal axis methods that try to improve rate of convergence on vertical axis, methods that try to overcome non-linearity and ill-conditioning

6 Dynamic sample size methods

Mini-batch stochastic gradient:

$$w_{k+1} \leftarrow w_k - |\alpha| g(w_k, \Xi_k)$$

where the stochastic directions are computed for some $\tau > 1$ as

$$g(w_k, \Xi_k) \stackrel{\text{def}}{=} \frac{1}{n_k} \sum \nabla f(w_k; \Xi_{k,i}) \quad \text{with } n_k \stackrel{\text{def}}{=} |\mathcal{S}_k| = \lceil \tau^{k-1} \rceil$$

Note that batch algorithm might be linearly convergent, and therefore faster than the regular gradient descent. It is Computationally slower.

6.1 Guidelines

- if the optimization method has a sublinear rate of convergence, then there is no sampling rate that makes the algorithm "efficient";
- if the optimization algorithm is linearly convergent, then the sampling rate must be geometric (with restrictions on the constant in the rate) for the algorithm to be "efficient"
- for superlinearly convergent methods, increasing the sample size at a rate that is slightly faster than geometric will yield an "efficient" method.

7 Gradient Aggregation

Rather than compute increasingly more **new** stochastic gradient information in each iteration, achieve a lower variance by **reusing and/or revising** previously computed information

This works on finite sums like R_n

SVRG: Method for Minimizing an Empirical Risk Rn

Procedure SVRG

Choose an initial iterate $w_1 \in \mathbb{R}^d$, setsize $\alpha > 0$ and a positive integer m;
for $k = 1, 2, \ldots$ **do**
  Compute the batch gradient $\nabla R_n(w_k)$
  Initialize $\tilde{w}_1 \leftarrow w_k$
  for $j = 1, \ldots, m$ **do**
    $\tilde{g}_j \leftarrow \nabla f_{i_j}(\tilde{g}_j) - \left(\nabla f_{i_j}(w_k) - \nabla R_n(w_k)\right)$;
    $\tilde{w}_{j+1} \leftarrow \tilde{w}_j - \alpha \tilde{g}_j$
  Option (a): Set $w_{k+1} = \tilde{w}_{m+1}$
  Option (b): Set $w_{k+1} = \frac{1}{m}\sum_{j=1}^{m} \tilde{w}_{j+1}$
  Option (c): Choose j uniformly from $\{1, \ldots, m\}$ and set $w_{k+1} = \tilde{w}_{j+1}$

SAGA: Method for Minimizing an Empirical Risk Rn

- Same per-iteration costs as basic SG

Procudure SAGA

Choose an initial iterate $w_1 \in \mathbb{R}^d$, setsize $\alpha > 0$;
for $i = 1, 2, \ldots, n$ **do**
  Compute $\nabla f_n(w_1)$
  Store $\nabla f_i\left(w_{[i]}\right) \leftarrow \nabla f_i(w_1)$
for $k = 1, 2 \ldots$ **do**
  Choose j uniformly in $\{1, \ldots, n\}$
  Compute $\nabla f_j(w_k)$;
  Set $g_k \leftarrow \nabla f_j(w_k) - \nabla f_j\left(w_{[j]}\right) + \frac{1}{n}\sum_{i=1}^{n} \nabla f_i\left(w_{[i]}\right)$
  Store $\nabla f_j\left(w_{[j]}\right) \leftarrow \nabla f_j\left(w_{[k]}\right))$
  Set $w_{k+1} \leftarrow w_k - \alpha g_k$

7.1 iterated Averaging Methods

$$w_{k+1} \leftarrow w_k - \alpha g(w_k, \Xi_k)$$

$$\tilde{w}_{k+1} \leftarrow \frac{1}{k+1}\sum_{i=1}^{k+1} w_i$$

For this $\tilde{w}_k$ has no effect in the computation of the SG iterate sequence